8

Advanced Goal-Based Programming

In this chapter, we focus on one aspect of ***Jason*** programming style: the use of *goals*. One of the most important aspects of programming multi-agent systems is in the *use* of goals. The simplest type of goal is an *achievement goal*: where we want an agent to achieve some particular state of affairs ('win the game'). *Maintenance goals* are similarly basic: with such goals, we want our agent to maintain some state of affairs ('ensure the reactor temperature stays below 100 degrees'). However, *combinations* of achievement and maintenance goals are also common: for example, we might want to maintain some state of affairs until some particular condition becomes true. AgentSpeak and ***Jason*** provide direct support for achievement goals: this is the basic '$!\phi$' subgoal construct of plans. However, richer goal structures are not directly supported. There has been much discussion of such *declarative goals* in the agent programming literature [23, 55, 83, 96, 98, 102], as well as the different types of commitments that agents can exhibit towards their goals. This is essential in giving agents what can be called *rational behaviour*, in the sense that we do not expect agents to give up achieving their goal unless it has become impossible to achieve or the motivation for the goal no longer exists, in which case we do not expect agents to pointlessly try to achieve their goal. We believe that providing explicit support for complex goal patterns of this kind will be a major advantage when developing many multi-agent applications.

In ***Jason***, we have avoided changing the language to explicitly support more complex goal structures. Instead, we use *pre-processing directives*, which transform plans according to certain patterns which then result in the goal behaviour of interest. The patterns as presented below were first introduced in [55]; we later show how the pre-processing directives can be used for each of the plan patterns. Before

Programming Multi-Agent Systems in AgentSpeak using Jason R.H. Bordini, J.F. Hübner, M. Wooldridge

we explain the plan patterns, we will comment on how the BDI notions map into ***Jason*** programming and some internal actions available in ***Jason*** to allow for what could be called 'BDI programming' or 'goal-based programming'.

8.1 BDI Programming

In Chapter 2, we saw that an idealised BDI agent would deliberate and then use means-ends reasoning to determine the best course of action to achieve its current goals. In practice, however, this is often too computationally expensive to be feasible for large applications. In Chapter 3, we saw that AgentSpeak uses the intuitive notions of the BDI architecture to provide high-level programming constructs, and let programmers do much of this job. However, it helps to have a clear picture of how the BDI notions map into programming constructs and data structures of the interpreter in ***Jason***. In Chapter 10 we give a more formal account of this relation. Before we start, it is worth recalling that the notions of *desire* and *goal* are very closely related. One normally uses 'goals' to refer to a consistent subset of an agent's 'desires'; in any case, both refer to states of affairs that the agent would like to bring about.

The notion of belief is straightforward as an agent in ***Jason*** has a belief base which contains a number of literals denoting the things directly believed by the agent. Intentions have two well-known aspects: they can be defined as the chosen course of action to achieve a goal (intention-to) or as the state that the chosen course of action aims to achieve (intention-that), in which case intentions are seen as a subset of the agent's desires, the ones it committed to bring about. The notion of intention-to is very clear in an AgentSpeak agent's intentional structure (see the 'set of intentions' in Figure 4.1): any intended means in that structure (i.e. an instance of a plan) is an intention-to. The intention-that notion, in particular in the declarative agent programming style that we discuss in this chapter, is naturally seen as the goal in the triggering event of a plan instance within the set of intentions. For example, if an agent has a plan '`+!g : b <- a.`' in its set of intentions, then `a` is an intention-to because `g` is an intention-that: the agent has the desire to achieve a state of affairs where `g` is true, and is committed to bring that about by being committed to executing `a`.

The notion of desire is a bit trickier. We just saw that intentions-that are a subset of desires. In particular, they are the specific desires for which the agent has chosen a course of action (and committed to execute it). These are the goals for which a plan is already in the set of intentions. Recall that, when new goals are created, they generate (internal) events that are included in the set of events (see Figure 4.1 again). At that point, the agent has a new goal (i.e. a new desire) but has not yet

committed to bringing it about; in fact, it may turn out that the agent might not have the know-how, the ability, or the resources to do so. While a relevant and applicable plan is not found for a particular goal currently in the set of intentions, we can refer to it as a *desire*, but it is certainly not an intention.

While in AgentSpeak the context part of a plan implicitly checks the agent's belief state, it does not allow for the agent's desires and intentions to be checked. Also, another fundamental part of a BDI agent is to *drop* goals, which is also not part of the AgentSpeak language. We have addressed these shortcomings through the use of some standard internal actions, as follows. Recall that standard internal actions, as opposed to user-defined internal actions, are those available with the ***Jason*** distribution; they are denoted by an action name starting with the '.' symbol. (We list all standard internal actions in Appendix A.3.)

`.desire`: this internal action takes a literal as parameter and succeeds if that literal is one of the agent's desires. In our framework, desires are achievement goals that appear in the set of events (i.e. internal events) or appear in intentions, including suspended ones. More specifically, the `.desire` action succeeds if the literal given as a parameter unifies with a literal that appears in a triggering event that has the form of a goal addition (i.e. `+!g`) within an event in the set of events or in the triggering event part of the head of any of the intended means (plans) in any of the agent's current intentions. One of the common views is that an intention is a desire (goal) which the agent is committed to achieving (by executing a plan, i.e. a course of action); goals in the set of events are those for which no applicable plan has become intended yet. Formal definitions are given in Section 10.4.

`.intend`: similar to `.desire`, but for an intention specifically (i.e. excluding the achievement goals that are internal events presently in the set of events).

`.drop_desire`: receives a literal as parameter and removes events that are goal additions with a literal that unifies with the one given as parameter, then does all that `.drop_intention` (described next) would do. That is, all intentions and events which would cause `.desire` to succeed for this argument are simply dropped.

`.drop_intention`: as `.drop_desire`, but works on each intention in the set of intentions where any of the goal additions in the triggering events of plans unify with the literal given as parameter. In other words, all intentions which would make `.intend` true are dropped.

`.drop_event`: this is complementary to the two internal actions above, to be used where only achievement goals appearing in events (desires that are not intentions) should be dropped.

`.drop_all_desires`: no parameters needed; all events and all intentions are dropped; this also includes external events (differently from the actions above).

`.drop_all_intentions`: no parameters needed; all intentions except the currently selected intention (the one where `.drop_all_intentions` appears) are dropped.

`.drop_all_events`: no parameters needed; all events (including external events) currently in the set of events are dropped.

We now take a closer look at two very important standard internal actions. They are particularly useful in combination with the plan failure handling mechanism introduced in Section 4.2, and are used in the plan patterns we introduce later in this chapter.

The internal actions often used in combination with plan failure are `.succeed_goal` and `.fail_goal`. The first, `.succeed_goal(`*g*`)`, is used when the agent realises the goal has already been achieved, so whatever plan was being executed to achieve that goal no longer needs to be executed. The second, `.fail_goal(`*g*`)`, is used when the agent realises that the goal has become impossible to achieve, therefore the plan that required *g* being achieved as one of its subgoals has to fail. More specifically, when `.succeed_goal(`*g*`)` is executed, any intention that has the goal *g* in the triggering event of any of its plans will be changed as follows. The plan with triggering event `+!`*g* is removed and the plan below that in the stack of plans forming that intention carries on being executed, at the point after goal *g* appeared in the plan body. Goal *g*, as it appears in the `.succeed_goal` internal action, is used to further instantiate the plan where the goal that was terminated early appears. With `.fail_goal(`*g*`)`, the plan for `+!`*g* is also removed, but an event for the deletion of the goal whose plan body required *g* is generated instead: as there is no way of achieving *g*, the plan requiring *g* to be achieved has effectively failed.

It is perhaps easier to see how these actions work with reference to Figure 8.1. The figure shows the consequence of each of these internal actions being executed. The plan where the internal action appeared is not shown; it is likely to be within another intention. Note that the state of the intentions, as shown in the figure, is not the immediate state resulting from the execution of either of the internal actions – i.e. not the state at the end of the reasoning cycle where the internal action was executed – but the most significant next state of the changed intention.

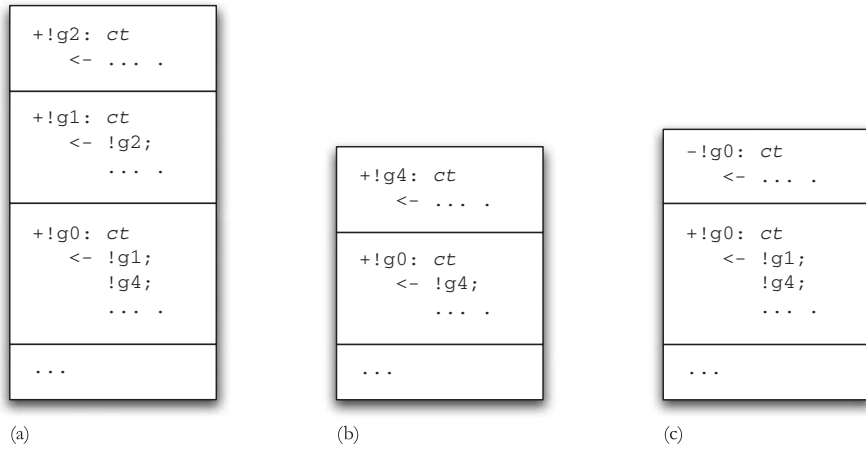

Figure 8.1 Standard internal actions for dropping goals. (a) Initial intention; (b) after `.succeed_goal(g1)`; (c) After `.fail_goal(g1)`.

8.2 Declarative (Achievement) Goal Patterns

Although goals form a central component of the AgentSpeak conceptual framework, it is important to note that the language itself does not provide any explicit constructs for handling goals with complex structures. As we noted above, a programmer will often think in terms of goals such as 'maintain *P* until *Q* becomes true', or 'prevent *P* from becoming true'. Creating AgentSpeak code to realise such complex goals can often be an essentially *ad hoc* process, dependent upon the experience of the programmer. A recent extension of ***Jason*** defined a number of declarative goal structures which can be realised in terms of *patterns* of AgentSpeak plans – that is, complex combinations of plan structures which are often useful in actual scenarios. Such patterns can be used to implement, in a systematic way, not only complex types of declarative goals, but also the types of agent commitments that they can represent, as discussed for example by Cohen and Levesque [29].

As an initial motivating example for declarative goals, consider a robot agent with the goal of being at some location (represented by the predicate `l(X,Y)`), and the following plan to achieve this goal:

```
+!l(X,Y) : bc(B) & B >0.2 <- go(X,Y).
```

where the predicate `bc/1` stands for 'battery charge', and `go` identifies an action that the robot is able to perform in the environment.

At times, using an AgentSpeak plan as a procedure can be quite useful as a programming practice. Thus, in a way, it is important that the AgentSpeak interpreter does not enforce any declarative semantics to its only (syntactically defined) goal construct. However, in the plan above, `l(X,Y)` is clearly meant as a declarative goal; that is, the programmer expects the robot to believe `l(X,Y)` (by perceiving the environment) if the plan executes to completion. If it fails because, say, the environment is dynamic, the goal cannot be considered achieved and, normally, should be attempted again.

*As much as possible, **Jason** programmers should bear in mind, when writing their code, that this is usually the right approach to thinking about* goals *that agents should achieve. While we find the freedom to use a goal simply as a procedure useful, it is by the declarative use of goals that we can determine whether some AgentSpeak code is using an appropriate style for agent programming. If you cannot find clearly declarative use of goals in your code, either your design is not appropriate, or your programming style is not right, or your application is not suitable for multi-agent systems.*

The type of situation in the example above is commonplace in multi-agent systems, and this is why it is important to be able to define declarative goals in agent-oriented programming. As pointed out by van Riemsdijk *et al.* [98], one can easily transform the above procedural goal into a declarative goal by adding a corresponding *test goal* at the end of the plan's body, as follows:

```
+!l(X,Y) : bc(B) & B >0.2 <- go(X,Y); ?l(X,Y).
```

This plan only succeeds if the goal is actually (believed to be) achieved; if the given (procedural) plan executes to completion (i.e. without failing) but the goal happens not to be achieved, the test goal at the end will fail. In this way, we have taken a simple *procedural* goal and transformed it into a *declarative* goal – the goal to achieve some state of affairs. We will see later that the plan failure mechanism in ***Jason*** can be used to account for the various attitudes that agents can have due to a declarative goal not being achieved (e.g. because the test goal at the end of the plan failed).

The above solution for simple declarative goals can be thought of as a *plan pattern*, which can be applied to solve other similar problems. This approach is inspired by the successful adoption of *design patterns* in object-oriented design [48].

To represent such patterns with AgentSpeak, we make use of skeleton programs with meta variables. For example, the general form of an AgentSpeak plan for a simple declarative goal, such as the one used in the robot's location goal above, is as follows:

```
+!g : c <- p; ?g.
```

Here, *g* is a meta variable that represents the declarative goal, *c* is a meta variable that represents the context expression stating in which circumstances the plan is

applicable, and p represents the procedural part of the plan body (i.e. a course of action to achieve g). Note that, with the introduction of the final test goal, this plan to achieve g finishes successfully only if the agent believes g after the execution of plan body p.

To simplify the presentation of the patterns, we also define pattern rules which rewrite a set of AgentSpeak plans into a new set of AgentSpeak plans according to a given pattern. The following pattern rule, called **DG** (declarative goal), is used to transform procedural goals into declarative goals. The pattern rule name is followed by the (subscript) parameters which need to be provided by the programmer, besides the actual code (i.e. a set of plans) on which the pattern will be applied.

```
+!g : c1 <- p1.
+!g : c2 <- p2.
...
+!g : cn <- pn.
```

DG$_g$ $(n \geq 1)$

```
+!g : g <- true.
+!g : c1 <- p1; ?g.
+!g : c2 <- p2; ?g.
...
+!g : cn <- pn; ?g.
+g : true <- .succeed_goal(g).
```

Essentially, this rule adds `?g` at the end of each plan in the given set of plans which have `+!g` as trigger event, and creates two extra plans (the first and the last plans above). The first plan checks whether the goal g has already been achieved – in such a case, there is nothing else to do. That last plan is triggered when the agent perceives that g has been achieved while it is executing any of the courses of action p_i $(1 \leq i \leq n)$ which aim at achieving g; in this circumstance, the plan being executed in order to achieve g can be immediately terminated. The internal action `.succeed_goal(g)` terminates such a plan with success (as explained in the previous section).

In this pattern, when one of the plans to achieve g fails, the agent gives up achieving the goal altogether. However it could be the case that, for such a goal, the agent should try another plan to achieve it, as in the 'backtracking' plan selection mechanism available in platforms such as JACK [101] and 3APL [35]. In those mechanisms, usually only when all available plans have been tried in turn and failed is the goal abandoned with failure, or left to be attempted again later on. The following rule, called **BDG** (backtracking declarative goal), defines this pattern based on a set of conventional AgentSpeak plans $\mathcal{P}$ transformed by the **DG** pattern (each plan in $\mathcal{P}$ is of the form `+!g : c <- p.`):

$\mathcal{P}$

BDG$_g$

DG$_g(\mathcal{P})$

```
-!g : true <- !!g.
```

The last plan of the pattern catches a failure event, caused when a plan from $\mathcal{P}$ fails, and then tries to achieve that same goal g again. Notice that it is possible that the same plan is selected and fails again, causing a loop if the plan contexts have not been carefully programmed. Therefore, the programmer would need to specify the plan contexts in such a way that a plan is only applicable if it has a chance of succeeding regardless of it having been tried already (recently).

Instead of worrying about defining contexts in such complex way, in some cases it may be useful for the programmer to apply the following pattern, called **EBDG** (exclusive backtracking declarative goal), which ensures that none of the given plans will be attempted twice before the goal is achieved:

```
+!g : c1 <- b1.
+!g : c2 <- b2.
...
+!g : cn <- bn.
```

EBDG$_g$

```
+!g : g <- true.
+!g : not p(1,g) & c1 <- +p(1,g); b1.
+!g : not p(2,g) & c2 <- +p(2,g); b2.
...
+!g : not p(n,g) & cn <- +p(n,g); bn.
-!g : true <- !!g.
 +g : true <- .abolish(p(_,g)); .succeed_goal(g).
```

In this pattern, each plan i, when selected for execution, initially adds a belief[1] `p(`i`,`g`)`; goal g is used as an argument to `p` so as to avoid interference between various instances of the pattern for different goals. The belief is used as part of the plan contexts (note the use of `not p(`i`,`g`)` in the contexts of the plans in the pattern above) to state that the plan should not be applicable in a second attempt (of that same plan within a single adoption of goal g for that agent).

8.3 Commitment Strategy Patterns

In the EBDG pattern, despite the various alternative plans, the agent can still end up dropping the intention with goal g unachieved, if all those plans become

[1] In the actual pattern implementation this predicate is slightly different to avoid name clash.

non-applicable. In BDI parlance, this is because the agent is not sufficiently *committed* to achieving the goal. We now introduce various patterns that can be used to program various forms of commitment strategies introduced in the BDI literature.

With a *blind commitment*, the agent can drop the goal only when it is achieved. This type of commitment towards the achievement of a declarative goal can thus be understood as *fanatical commitment* [81]. The $\mathbf{BC}_{g,F}$ pattern below defines this type of commitment:

$\mathcal{P}$

$$\frac{}{\mathbf{F}(\mathcal{P})}\ \mathbf{BC}_{g,F}$$

```
+!g : true <- !!g.
```

This pattern is based on another pattern rule, represented by the variable $\mathbf{F}$; $\mathbf{F}$ is often **BDG**, although the programmer can chose any other pattern (e.g. **EBDG** if a plan should not be attempted twice). Finally, the last plan makes the agent attempt to achieve the goal even in case where there is no applicable plan. It is assumed that the selection of plans is based on the order that the plans appear in the program and all events have equal chance of being chosen as the event to be handled in a reasoning cycle. If the selection functions have been customised, this pattern needs to be used with care or changed, unless the customisation takes into consideration the use of this pattern.

For most applications, **BC**-style fanatical commitment is too strong. For example, if a robot has the goal to be at some location, it is reasonable that it can drop this goal if its battery charge is getting very low; in other words, the agent has realised that it has become impossible to achieve the goal, so it is useless to keep attempting it. This is very similar to the idea of a persistent goal in the work of Cohen and Levesque: a persistent goal is a goal that is maintained as long as it is believed not achieved, but still believed possible [29]. In [102] and [23], the 'impossibility' condition is called 'drop condition'. The drop condition f (e.g. 'low battery charge') is used in the single-minded commitment (**SMC**) pattern to allow the agent to drop a goal if it becomes impossible:

$\mathcal{P}$

$$\frac{}{\mathbf{BC}_{g,BDG}(\mathcal{P})}\ \mathbf{SMC}_{g,f}$$

```
+f : true <- .fail_goal(g).
```

This pattern extends the **BC** pattern adding the drop condition represented by the literal f in the last plan. If the agent comes to believe f, it can drop goal g, signalling failure (refer to the semantics of the internal action `.fail_goal` in

the previous section). This effectively means that the plan in the intention where g appeared, which depended on g being achieved to then carry on the plan execution, must itself fail (as g is now impossible to achieve). However, there might be an alternative for that plan which does not depend on g, so that plan's failure handling may take care of such situation.

As we have a failure drop condition for a goal, we can also have a success drop condition, e.g., because the motivation to achieve the goal has ceased to exist. Suppose a robot has the goal of going to the fridge because its owner has asked it to fetch a beer from there; then, if the robot realises that its owner does not want a beer anymore, it should drop the goal [29]. The belief 'my owner wants beer' is the *motivation* (m) for the goal. The following pattern, called relativised commitment (**RC**) defines a goal that is relative to a motivation condition: the goal can be dropped with success if the agent no longer has the motivation for it.

$\mathcal{P}$

$$\frac{}{\mathbf{BC}_{g,BDG}(\mathcal{P})} \; \mathbf{RC}_{g,m}$$

$-m$ `: true <- .succeed_goal(`g`).`

Note that, in the particular combination of **RC** and **BC** above, if the attempt to achieve g ever terminates, it will always terminate with success, since the goal will be dropped only if either the agent believes it has been achieved (by **BC**) or m is removed from belief base.

Of course we can combine the last two patterns above to create a goal which can be dropped if it has been achieved, has become impossible to achieve, or the motivation to achieve it no longer exists, representing what is called an 'open-minded commitment'. The open-minded commitment pattern (**OMC**) defines this type of goal:

$\mathcal{P}$

$$\frac{}{\mathbf{BC}_{g,BDG}(\mathcal{P})} \; \mathbf{OMC}_{g,f,m}$$

$+f$ `: true <- .fail_goal(`g`).`
$-m$ `: true <- .succeed_goal(`g`).`

For example, an impossibility condition could be 'no beer at location (X,Y)' (denoted below by `~b(X,Y)`), and the motivation condition could be 'my owner wants a beer' (denoted below by `wb`). Consider the plan below as representing the single known course of action to achieve goal `l(X,Y)`:

```
+!l(X,Y) : bc(B) & B > 0.2 <- go(X,Y).
```

When the pattern $\mathbf{OMC}_{l(X,Y),\sim b(X,Y),wb}$ is applied to the initial plan above, we get the following set of plans:

```
+!l(X,Y) : l(X,Y) <- true.
+!l(X,Y) : bc(B) & B > 0.2 <- go(X,Y); ?l(X,Y).
-!l(X,Y) : true <- !!l(X,Y).
+!l(X,Y) : true <- !!l(X,Y).
+~b(X,Y) : true <- .fail_goal(l(X,Y)).
-wb : true <- .succeed_goal(l(X,Y)).
```

8.4 Other Useful Patterns

Another important type of goal in agent-based systems are *maintenance goals*, whereby an agent needs to ensure that the state of the world will always be such that g holds. Such an agent will need plans to act on the events that indicate the maintenance goal may fail in the future. In realistic environments, however, agents will probably fail in preventing the maintenance goal from ever failing. Whenever the agent realises that g is no longer in its belief base (i.e. believed to be true), it will certainly attempt to bring about g again by having the respective (declarative) achievement goal. The pattern rule that defines a maintenance goal (**MG**), but particularly in the sense of realising the failure in a goal maintenance, is as follows:

$\mathcal{P}$

——————————— $\mathbf{MG}_{g,F}$

```
g[source(percept)].
-g : true <- !g.
```

$\boldsymbol{F}(\mathcal{P})$

The first line of the pattern states that, initially (when the agent starts running) it will assume that g is true. (As soon as the interpreter obtains perception of the environment for the first time, the agent might already realise that such assumption was wrong.) The first plan is triggered when g is removed from the belief base, e.g. because g has not been perceived in the environment in a given reasoning cycle, and thus the maintenance goal g is no longer achieved. This plan then creates a declarative goal to achieve g. The type of commitment to achieving g if it happens not to be true is defined by $\boldsymbol{F}$, which would normally be **BC** given that the goal should not be dropped in any circumstances unless it is has been achieved again. (Realistically, plans for the agent to attempt pro-actively to prevent this from ever happening would also be required, but the pattern is useful to make sure the agent will act appropriately if things go wrong.)

We now show another useful pattern, called sequenced goal adoption (**SGA**). This pattern should be used when various instances of a plan to achieve a goal should not be adopted concurrently (e.g. a robot should not try to clean two different places at the same time, even if it has perceived dirt in both places, which will lead to the adoption of goals to clean both places). To solve this problem,

the **SGA** pattern adopts the first occurrence of the goal and records the remaining occurrences as pending goals by adding them as special beliefs. When one such goal occurrence is achieved, if any other occurrence is pending, it gets activated.

SGA$_{t,c,g}$

```
t : not fl(_) & c <- !fg(g).
t : fl(_) & c <- +fl(g).
+!fg(g) : true <- +fl(g); !g; -fl(g).
-!fg(g) : true <- -fl(g).
-fl(_) : fl(g) <- !fg(g).
```

In this pattern, *t* is the trigger leading to the adoption of a goal *g*; *c* is the context for the goal adoption; `fl(g)` is the flag to control whether an instance of goal *g* is already active; and `fg(g)` is a procedural goal that guarantees that `fl` will be added to the belief base to record the fact that some instance of the goal has already been adopted, then adopts the goal `!g`, as well as guaranteeing that `fl` will be eventually removed whether `!g` succeeds or not. The first plan is selected when *g* is not being pursued; it simply calls the `fg` goal. The second plan is used if some other instance of that goal has already been adopted. All it does is to remember that this goal *g* was not immediately adopted by adding `fl(g)` to the belief base. The last plan makes sure that, whenever a goal adoption instance is finished (denoted by the deletion of an `fl` belief), if there are any pending goal instances to be adopted, they will be activated through the `fg` call.

8.5 Pre-Processing Directives for Plan Patterns

Once the above goal and commitment strategies type are well understood, it is very easy to implement them in ***Jason*** using the pre-processing directives for the various plan patterns. They all have a `{begin ...} ... {end}` structure. The plan-pattern directives currently available in ***Jason*** are as follows:

`{begin dg(`⟨*goal*⟩`)}`
⟨*plans*⟩
`{end}`

This is the 'declarative goal' pattern described above.

`{begin bdg(`⟨*goal*⟩`)}`
⟨*plans*⟩
`{end}`

This is the 'backtracking declarative goal' pattern described above. Recall that further patterns can be used in the body of the directive where the ⟨*plans*⟩ are given.

```
{begin ebdg(⟨goal⟩)}
   ⟨plans⟩
{end}
```

This is the 'exclusive backtracking declarative goal' pattern described above.

```
{begin bc(⟨goal⟩,⟨F⟩)}
   ⟨plans⟩
{end}
```

This is used to add a 'blind commitment' strategy to a given goal. *F* is the name of another pattern to be used for the goal itself, the one for which the commitment strategy will be added; it is typically **BDG**, but could be any other goal type.

```
{begin smc(⟨goal⟩,⟨f⟩)}
   ⟨plans⟩
{end}
```

This is the pattern to add a 'single-minded commitment' strategy to a given goal; *f* is the *failure condition*.

```
{begin rc(⟨goal⟩,⟨m⟩)}
   ⟨plans⟩
{end}
```

This is the pattern to add a 'relativised commitment' strategy to a given goal; *m* is the *motivation* for the goal.

```
{begin omc(⟨goal⟩,⟨f⟩,⟨m⟩)}
   ⟨plans⟩
{end}
```

This is the pattern to add an 'open-minded commitment' strategy to a given goal; *f* and *m* are as in the previous two patterns.

```
{begin mg(⟨goal⟩,⟨F⟩)}
   ⟨plans⟩
{end}
```

This is the 'maintenance goal' pattern; *F* is the name of another pattern to be used for the achievement goal, in case the maintenance goal fails. Recall this is a reactive form of a maintenance goal, rather than proactive. It causes the agent to act to return to a desired state after things have already gone wrong.

```
{begin sga(⟨t⟩,⟨c⟩,⟨goal⟩)}
   ⟨plans⟩
{end}
```

This is the 'sequenced goal adoption' pattern; the pattern prevents multiple instances of the same plan to be simultaneously adopted, where *t* is the triggering event and *c* is the context of the plan.